# SAA - ACTIVIDAD 3.16 - ÁLVARO FERNANDEZ BECERRA

Resuelve el problema de clasificación de las enfermedades del corazón con un clasificador RandomForestClassifier sin especificar valor para ningún parámetro para el constructor, está bien con los valores por defecto.

Utiliza validación simple con un 80% de datos para entrenamiento.

Muestra un gráfico con la relevancia de los atributos para el proceso de clasificación, ordenados por relevancia.

Una vez hecho esto, utiliza GridSearchCV para encontrar los valores óptimos para los parámetros del constructor (hiperparámetros).

Prueba valores alternativos para max_features, para que no utilice todos los atributos predictores.

Nota: Al hacer GridSearchCV no hace falta probar con tantos hiperparámetros como con DecisionTreeClasiffier, puedes centrarte en los que consideres más importantes.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"

# Nombres de columnas
columnas = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'
]

df = pd.read_csv(url, header=None, names=columnas, na_values='?')
df.head()
```

| | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | num |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 63.0 | 1.0 | 1.0 | 145.0 | 233.0 | 1.0 | 2.0 | 150.0 | 0.0 | 2.3 | 3.0 | 0.0 | 6.0 | 0 |
| **1** | 67.0 | 1.0 | 4.0 | 160.0 | 286.0 | 0.0 | 2.0 | 108.0 | 1.0 | 1.5 | 2.0 | 3.0 | 3.0 | 2 |
| **2** | 67.0 | 1.0 | 4.0 | 120.0 | 229.0 | 0.0 | 2.0 | 129.0 | 1.0 | 2.6 | 2.0 | 2.0 | 7.0 | 1 |
| **3** | 37.0 | 1.0 | 3.0 | 130.0 | 250.0 | 0.0 | 0.0 | 187.0 | 0.0 | 3.5 | 3.0 | 0.0 | 3.0 | 0 |
| **4** | 41.0 | 0.0 | 2.0 | 130.0 | 204.0 | 0.0 | 2.0 | 172.0 | 0.0 | 1.4 | 1.0 | 0.0 | 3.0 | 0 |

```
#df.info()
df.isnull().sum()
```

| | 0 |
|---|---|
| **age** | 0 |
| **sex** | 0 |
| **cp** | 0 |
| **trestbps** | 0 |
| **chol** | 0 |
| **fbs** | 0 |
| **restecg** | 0 |
| **thalach** | 0 |
| **exang** | 0 |
| **oldpeak** | 0 |
| **slope** | 0 |
| **ca** | 4 |
| **thal** | 2 |
| **num** | 0 |

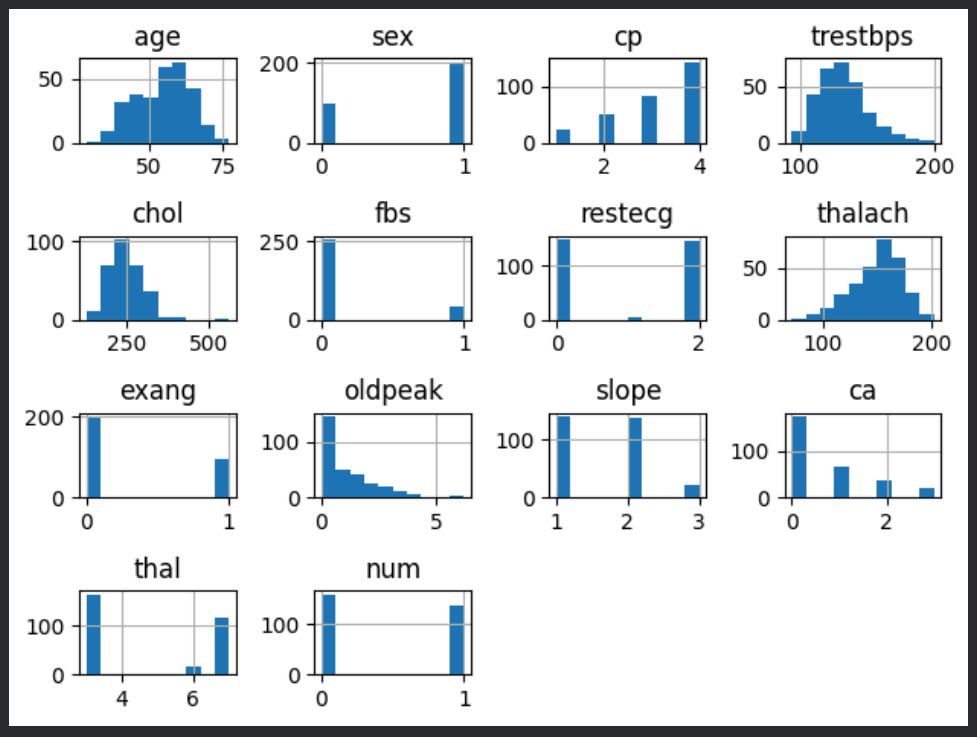
**dtype:** int64

```
# Eliminar filas con nulos
df.dropna(inplace=True)

# Convertimos num a binario donde: 0:sano y valores mayores -> 1:enfermo
for ind, row in df.iterrows():
    if row['num'] > 0:
        df.at[ind, 'num'] = 1

# x y
x = df.drop(['num', 'cp', 'restecg', 'slope', 'thal'], axis=1)
y = df['num']
df.dtypes
```

| | 0 |
|---|---|
| age | float64 |
| sex | float64 |
| cp | float64 |
| trestbps | float64 |
| chol | float64 |
| fbs | float64 |
| restecg | float64 |
| thalach | float64 |
| exang | float64 |
| oldpeak | float64 |
| slope | float64 |
| ca | float64 |
| thal | float64 |
| num | int64 |

**dtype:** object

```
df.hist()
plt.tight_layout()
```

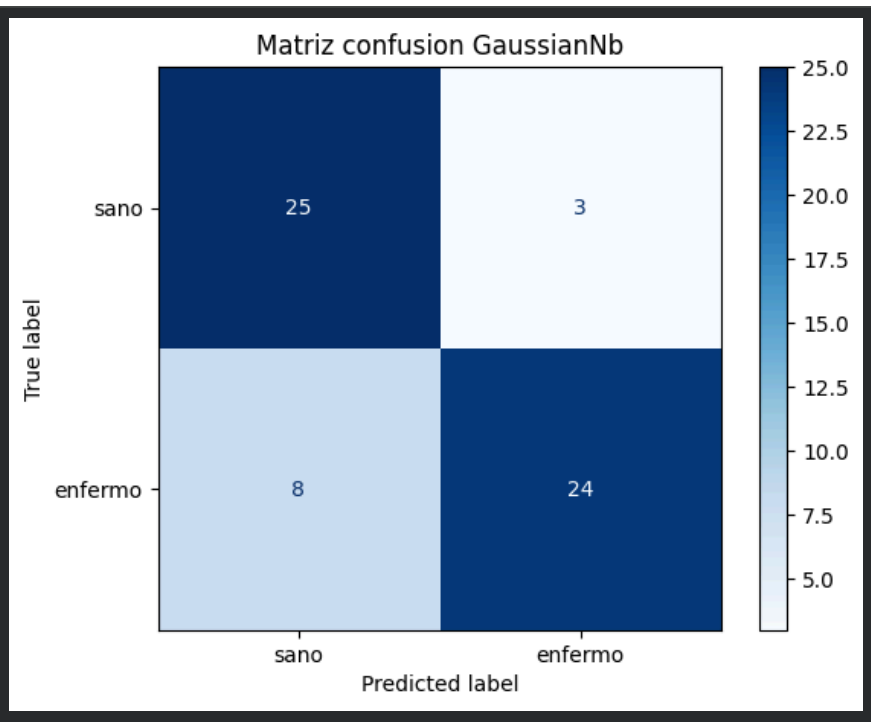



## 1. Clasificador GaussianNB:

```
RANDOM_SEED = 33

x_train, x_test, y_train, y_test = train_test_split(
    x, y, train_size=0.8, random_state=RANDOM_SEED
)

gau = GaussianNB()
gau.fit(x_train, y_train)

y_pred_gau = gau.predict(x_test)

print("Resultados: ")
print(f"Accuracy: {accuracy_score(y_test, y_pred_gau)}")
print(classification_report(y_test, y_pred_gau, target_names=['sano', 'enfermo']))
```

```
Resultados:
Accuracy: 0.8166666666666667
              precision    recall  f1-score   support

        sano       0.76      0.89      0.82        28
     enfermo       0.89      0.75      0.81        32

    accuracy                           0.82        60
   macro avg       0.82      0.82      0.82        60
weighted avg       0.83      0.82      0.82        60
```

```
cm_gau = confusion_matrix(y_test, y_pred_gau)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_gau, display_labels=['sano', 'enfermo'])
disp.plot(cmap='Blues')
plt.title('Matriz confusion GaussianNb')
plt.show()
```




## 2. RANDOMFOREST:

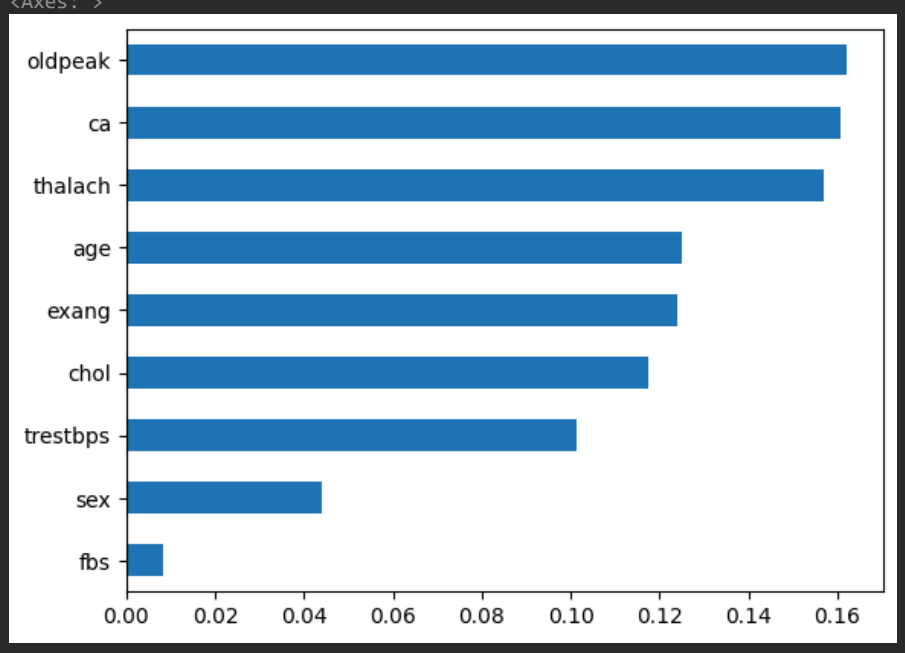
```
clf = RandomForestClassifier(random_state=RANDOM_SEED)

clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

print("accuray score: ", accuracy_score(y_test, y_pred))
```

```
accuray score:  0.7333333333333333
```

### Importancia de atributos (antes de GridSearch):

```
pd.Series(clf.feature_importances_, index=clf.feature_names_in_).nlargest(clf.feature_names_in_.shape[0]).sort_values(ascending=True).plot(kind='barh')
```

```
<Axes: >
```




### Aplicar GridSearch:

```
param_dist = {
  #'criterion':          ['gini', 'entropy'],
  'min_samples_split':  [1, 2, 4, 8],  # Por defecto: 2
  'min_samples_leaf':   [1, 2, 4, 8],  # Por defecto: 1
  'max_depth':          [None, 4, 8],
  'n_estimators':       [100, 200, 300, 400],
  'max_features':       [None, 'sqrt', 'log2']
}
```

```
grid_clf = GridSearchCV(estimator = RandomForestClassifier(random_state = RANDOM_SEED), param_grid = param_dist, cv = 5)

grid_clf.fit(x, y)
```

Mostrar salida oculta

```
print(grid_clf.best_params_)
print(grid_clf.best_score_)
```

```
{'max_depth': 4, 'max_features': None, 'min_samples_leaf': 8, 'min_samples_split': 2, 'n_estimators': 400}
0.8113559322033899
```

### Entrenamiento y validación simple utilizando el mejor clasificador de GridSearch:

```
clf = grid_clf.best_estimator_

clf.fit(x_train, y_train)
```

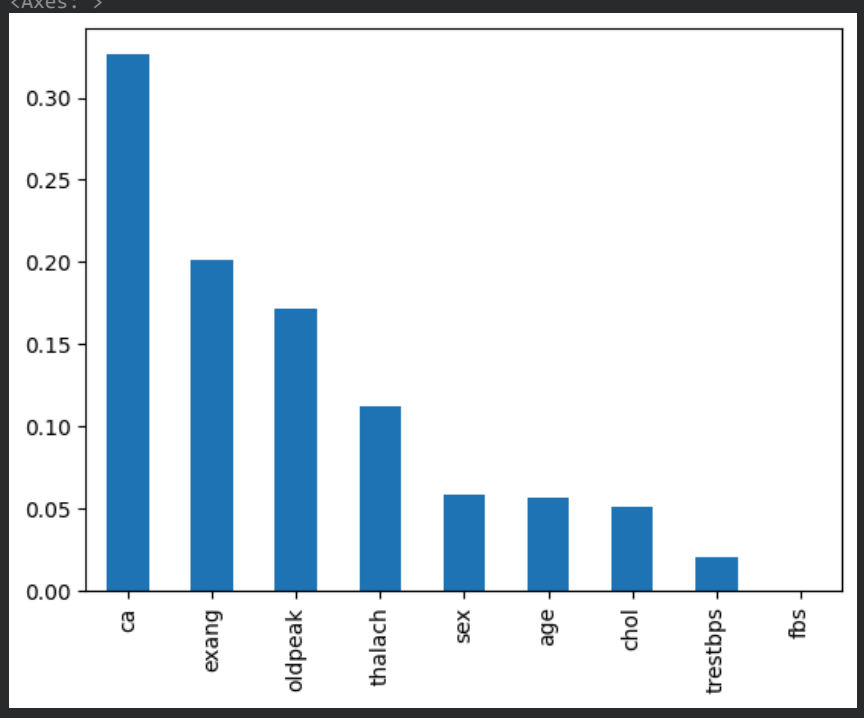
```
RandomForestClassifier(max_depth=4, max_features=None, min_samples_leaf=8,
                       n_estimators=400, random_state=33)
```

```
y_pred = clf.predict(x_test)

print("accuray score: ", accuracy_score(y_test, y_pred))
```

```
accuray score:  0.7833333333333333
```

```
pd.Series(clf.feature_importances_, index=clf.feature_names_in_).nlargest(clf.feature_names_in_.shape[0]).plot(kind='bar')
```

```
<Axes: >
```




- `ca` pasa a ser la variable con mayyor relevancia.